FinGraph — Week 1 & Week 2

Neo4j Browser Graph Queries — Day-by-Day Reference

Use these queries in Neo4j Browser and select Graph view when the query returns nodes and relationships. These are visualization queries; they do not delete data.

WEEK 1 — NEO4J GRAPH QUERIES

Day 1 — Basic Graph

```
MATCH (n)-[r]->(m)
RETURN n,r,m
LIMIT 200;
```

```
MATCH (p:Person)-[:OWNS]->(a:Account)
MATCH (b:Bank)-[:HOSTS]->(a)
RETURN p,a,b;
```

Day 2 — Ownership / Banks

```
MATCH (p:Person)-[:OWNS]->(a:Account)
RETURN p,a;
```

```
MATCH (b:Bank)-[:HOSTS]->(a:Account)
RETURN b,a;
```

Day 3 — Transactions

```
MATCH (a:Account)-[:SENDS]->(t:Transaction)-[:TRANSFERRED_TO]->(b:Account)
RETURN a,t,b
LIMIT 200;
```

```
MATCH (t:Transaction)
RETURN t
ORDER BY t.timestamp;
```

Day 4 — Transfers

```
MATCH (a:Account)-[:SENDS]->(t:Transaction)-[:TRANSFERRED_TO]->(b:Account)
RETURN a,t,b
LIMIT 200;
```

```
MATCH (a:Account)-[:SENDS]->(t:Transaction)-[:TRANSFERRED_TO]->(b:Account)
RETURN a.account_id AS Source,
       b.account_id AS Destination,
       t.transaction_id AS Transaction,
       t.amount AS Amount;
```

Day 5 — Transaction Investigation

```
MATCH (a:Account)-[:SENDS]->(t:Transaction)
RETURN a,t
LIMIT 100;
```

```
MATCH (t:Transaction)-[:TRANSFERRED_TO]->(a:Account)
RETURN t,a
LIMIT 100;
```

Day 6 — Financial Graph

```
MATCH (a:Account)-[:SENDS]->(t:Transaction)-[:TRANSFERRED_TO]->(b:Account)
RETURN a,t,b
LIMIT 200;
```

```
MATCH (a:Account)-[:SENDS]->(t:Transaction)-[:TRANSFERRED_TO]->(b:Account)
RETURN a.account_id AS Source,
       b.account_id AS Destination,
       t.amount AS Amount
ORDER BY Amount DESC;
```

Day 7 — Final Graph / Counts

```
MATCH (n)-[r]->(m)
RETURN n,r,m
LIMIT 500;
```

```
MATCH (n)
RETURN labels(n) AS NodeType,
       count(n) AS Count;
```

```
MATCH ()-[r]->()
RETURN type(r) AS RelationshipType,
       count(r) AS Count;
```

WEEK 2 — NEO4J GRAPH QUERIES

Day 1 — Flink Pipeline

```
MATCH (a:Account)-[r1:SENDS]->(t:Transaction)-[r2:TRANSFERRED_TO]->(b:Account)
RETURN a,r1,t,r2,b
LIMIT 200;
```

```
MATCH (n)
RETURN labels(n) AS NodeType,
       count(n) AS Count;
```

Day 2 — Kafka Transactions

```
MATCH (a:Account)-[:SENDS]->(t:Transaction)-[:TRANSFERRED_TO]->(b:Account)
RETURN a.account_id AS Source,
       t.transaction_id AS Transaction,
       t.amount AS Amount,
       b.account_id AS Destination
ORDER BY t.timestamp DESC
LIMIT 100;
```

Day 3 — Validation / Suspicious

```
MATCH (a:Account)-[:SENDS]->(t:Transaction)-[:TRANSFERRED_TO]->(b:Account)
WHERE t.is_suspicious = true
RETURN a,t,b
LIMIT 100;
```

```
MATCH (a:Account)-[:SENDS]->(t:Transaction)-[:TRANSFERRED_TO]->(b:Account)
WHERE t.is_suspicious = true
RETURN a.account_id AS Source,
       t.transaction_id AS Transaction,
       t.amount AS Amount,
       b.account_id AS Destination,
       t.is_suspicious AS Suspicious
ORDER BY Amount DESC;
```

Day 4 — Idempotent Updates

```
MATCH (a:Account)-[:SENDS]->(t:Transaction)-[:TRANSFERRED_TO]->(b:Account)
RETURN a,t,b
LIMIT 200;
```

```
MATCH (t:Transaction)
WITH t.transaction_id AS transaction_id, count(t) AS count
WHERE count > 1
RETURN transaction_id, count;
```

Day 5 — Fraud Detection

```
MATCH (a:Account)-[:SENDS]->(t:Transaction)-[:TRANSFERRED_TO]->(b:Account)
RETURN a,t,b
LIMIT 100;
```

```
MATCH (a:Account)-[:SENDS]->(t1:Transaction)-[:TRANSFERRED_TO]->(b:Account)
MATCH (b)-[:SENDS]->(t2:Transaction)-[:TRANSFERRED_TO]->(c:Account)
WHERE a <> b AND b <> c AND a <> c
  AND t1.timestamp <= t2.timestamp
RETURN a,t1,b,t2,c
LIMIT 50;
```

```
MATCH (a:Account)-[:SENDS]->(t1:Transaction)-[:TRANSFERRED_TO]->(b:Account)
MATCH (b)-[:SENDS]->(t2:Transaction)-[:TRANSFERRED_TO]->(c:Account)
MATCH (c)-[:SENDS]->(t3:Transaction)-[:TRANSFERRED_TO]->(d:Account)
WHERE a <> b AND b <> c AND c <> d
  AND a <> c AND a <> d AND b <> d
  AND t1.timestamp <= t2.timestamp AND t2.timestamp <= t3.timestamp
RETURN a,t1,b,t2,c,t3,d
LIMIT 50;
```

```
MATCH (src:Account)-[:SENDS]->(t:Transaction)-[:TRANSFERRED_TO]->(hub:Account)
WITH hub, count(DISTINCT src) AS senders,
     collect(DISTINCT src.account_id) AS sender_accounts,
     sum(t.amount) AS total_amount
WHERE senders >= 3
RETURN hub,senders,sender_accounts,total_amount
ORDER BY senders DESC;
```

Day 6 — Circular Flow / Risk

```
MATCH (a:Account)-[:SENDS]->(t1:Transaction)-[:TRANSFERRED_TO]->(b:Account)
MATCH (b)-[:SENDS]->(t2:Transaction)-[:TRANSFERRED_TO]->(c:Account)
MATCH (c)-[:SENDS]->(t3:Transaction)-[:TRANSFERRED_TO]->(a)
WHERE a <> b AND b <> c AND a <> c
  AND t1.timestamp <= t2.timestamp AND t2.timestamp <= t3.timestamp
RETURN a,t1,b,t2,c,t3
LIMIT 50;

MATCH (a:Account)
WHERE a.risk_score IS NOT NULL
RETURN a;

MATCH (a:Account)
WHERE a.risk_score IS NOT NULL
RETURN a.account_id AS Account,
       a.risk_score AS RiskScore,
       a.risk_level AS RiskLevel
ORDER BY RiskScore DESC;
```

Day 7 — Final Week 2 Graph

```
MATCH (n)-[r]->(m)
RETURN n,r,m
LIMIT 500;

MATCH (a:Account)-[:SENDS]->(t:Transaction)-[:TRANSFERRED_TO]->(b:Account)
RETURN a,t,b
LIMIT 500;

MATCH (n)
RETURN labels(n) AS NodeType,
       count(n) AS Count;

MATCH ()-[r]->()
RETURN type(r) AS RelationshipType,
       count(r) AS Count;
```

MOST IMPORTANT DEMO QUERIES

```
MATCH (n)-[r]->(m)
RETURN n,r,m
LIMIT 500;

MATCH (a:Account)-[:SENDS]->(t:Transaction)-[:TRANSFERRED_TO]->(b:Account)
RETURN a,t,b
LIMIT 200;

MATCH (a:Account)-[:SENDS]->(t1:Transaction)-[:TRANSFERRED_TO]->(b:Account)
MATCH (b)-[:SENDS]->(t2:Transaction)-[:TRANSFERRED_TO]->(c:Account)
WHERE a <> b AND b <> c AND a <> c
RETURN a,t1,b,t2,c
LIMIT 50;

MATCH (a:Account)-[:SENDS]->(t1:Transaction)-[:TRANSFERRED_TO]->(b:Account)
MATCH (b)-[:SENDS]->(t2:Transaction)-[:TRANSFERRED_TO]->(c:Account)
MATCH (c)-[:SENDS]->(t3:Transaction)-[:TRANSFERRED_TO]->(d:Account)
WHERE a <> b AND b <> c AND c <> d
  AND a <> c AND a <> d AND b <> d
RETURN a,t1,b,t2,c,t3,d
LIMIT 50;

MATCH (a:Account)-[:SENDS]->(t1:Transaction)-[:TRANSFERRED_TO]->(b:Account)
MATCH (b)-[:SENDS]->(t2:Transaction)-[:TRANSFERRED_TO]->(c:Account)
MATCH (c)-[:SENDS]->(t3:Transaction)-[:TRANSFERRED_TO]->(a)
WHERE a <> b AND b <> c AND a <> c
RETURN a,t1,b,t2,c,t3
LIMIT 50;
```